

■ BOND APP

Complete Technical Guide

Everything you built, every error you hit, every fix you applied — explained simply.

Project	Bond — AI Friendship Companion App
Stack	React · Node.js · Express · SQLite · Google Gemini AI
Hosting	Vercel (frontend) + Render (backend)
AI Model	Google Gemini 1.5 Flash (free tier)
Author	Built by you with AI assistance

Table of Contents

PART 1 — What Is The Bond App?

- › The idea & problem it solves
- › What each feature does

PART 2 — The Tech Stack — What & Why

- › Frontend: React + Vite
- › Backend: Node.js + Express
- › Database: SQLite
- › AI: Google Gemini
- › Auth: JWT tokens
- › Hosting: Vercel + Render

PART 3 — How The Code Is Structured

- › Folder structure explained
- › How frontend talks to backend
- › How the database works
- › How authentication works
- › How AI is called

PART 4 — Every Error & How We Fixed It

- › Error 1: Unexpected token 'T' — JSON parse error
- › Error 2: Node.js version incompatibility
- › Error 3: Failed to fetch — CORS error on Vercel
- › Error 4: Wrong Vercel URL in CORS allowlist

PART 5 — Deployment — How It All Works Online

- › GitHub — what it is and why we use it
- › Render — how the backend stays alive
- › Vercel — how the frontend is served
- › Environment variables explained

PART 6 — Questions Anyone Might Ask You

- › 20+ Q&A; covering all common questions

PART 1

What Is The Bond App?

Bond is a web application built for two close friends. Think of it as a private, AI-powered space that belongs to just the two of you — not a social network with hundreds of people, just you and one friend.

The Problem It Solves

Despite having WhatsApp, Instagram and Snapchat, close friendships are actually declining worldwide. Studies show that 1 in 4 adults feel chronically lonely. People don't drift apart because they stop caring — they drift apart because life gets busy and there is no dedicated space to *actively* nurture a deep friendship. Bond fixes that.

The Five Features — What Each One Does

■ Shared Memories

A private photo/emoji scrapbook. Both friends can add memories with a title, description, date and emoji or image URL. Only the two of you can ever see them. Like a shared diary.

■ Bucket List

A shared to-do list of experiences you want to have together — hiking, cooking, travelling, watching films. Either friend can add items. Either friend can mark them as done. The app tracks how many you've completed.

■ Daily Check-in

Each friend picks a mood (1–5 scale) and optionally writes a message about their day. The AI reads your friendship history — how many days bonded, memories, mood trends — and writes a personal, warm response. Not generic. Specific to your bond.

■ Support Mode

When your friend is going through something hard (breakup, exam stress, family issues), you type the situation and the AI gives you 4–5 specific, actionable ideas for how to show up for them. Removes the 'I don't know what to say' problem.

■ Bond Insights

The AI looks at all your data — days bonded, memories count, bucket list progress, average mood — and writes a short poetic insight about what makes your friendship special, plus two suggestions for the week.

How Two Friends Use It

- **Step 1:** Friend A opens the Vercel link and creates an account.
- **Step 2:** They see their 8-character invite code (e.g. AB12CD34) on the dashboard.
- **Step 3:** Friend A sends this code to Friend B on WhatsApp.
- **Step 4:** Friend B opens the same Vercel link, creates their own account, enters the invite code.
- **Step 5:** They are now 'bonded' — all five features unlock and they share the same private space.

PART 2

The Tech Stack — What & Why

A 'tech stack' is just the collection of technologies used to build something. Think of it like the ingredients in a recipe. Here is every ingredient in the Bond app and the reason it was chosen.

■■ Frontend: React + Vite

What is it?	React is a JavaScript library made by Facebook/Meta for building user interfaces — the part of the app you see and click on.
What is Vite?	Vite is the tool that bundles and serves the React code during development. Think of it as the engine that runs React on your computer.
Why React?	It lets you build the UI in small reusable pieces called 'components'. For example, the Sidebar is one component, each Memory Card is another. This makes the code organised and easy to update.
Key files	App.jsx (routing), AuthContext.jsx (global login state), pages/ (each screen), index.css (all styling)

■■ Backend: Node.js + Express

What is Node.js?	Node.js lets you run JavaScript on the server (your computer or Render). Normally JavaScript only runs in browsers — Node.js changed that.
What is Express?	Express is a framework that sits on top of Node.js and makes it easy to create API endpoints — the URLs the frontend calls to get or save data.
Why these?	They are the most popular combination for beginners. Huge community, tons of tutorials, runs everywhere.
What it does	Handles all requests: register a user, log in, save a memory, call the AI, complete a bucket list item, etc.
Key files	server.js (main entry point), routes/auth.js, routes/memories.js, routes/bucketlist.js, routes/ai.js

■■ Database: SQLite (via better-sqlite3)

What is it?	SQLite is a database — it stores data permanently in a file called bond.db. It's like a very organised Excel spreadsheet that code can read and write.
Why SQLite?	It requires zero setup. No separate server, no account. The database is just one file. Perfect for a personal project.
What's in it?	5 tables: users, memories, bucket_list, checkins, messages. Each table is like a sheet in Excel with rows and columns.
bond_id	When two friends bond, we create a unique ID like '3_7' (smaller user ID first, larger second). All memories and bucket items use this ID so only the right pair can see them.

■ AI: Google Gemini 1.5 Flash

What is it?	Gemini is Google's AI model — similar to ChatGPT but made by Google. We use the 'Flash' version which is fast and free.
Why Gemini?	Completely free. 15 requests per minute, 1 million tokens per day. More than enough for a personal friendship app. Originally we used Anthropic's Claude API but switched to Gemini to keep costs at zero.
How it's used	Three AI endpoints: /api/ai/checkin (daily response), /api/ai/support (how to help friend), /api/ai/insights (bond overview).
The key	Get your free API key at aistudio.google.com/apikey — sign in with Google, one click, done.

■ Authentication: JWT Tokens

What is JWT?	JSON Web Token. When you log in, the server creates a special encrypted string (the token) and sends it to your browser. Your browser saves it and sends it with every future request to prove you are who you say you are.
Think of it as	A wristband at a concert. You prove your ticket once at the door, get a wristband, then show the wristband to get into different areas. JWT is that wristband for your app.
bcrypt	Before saving your password to the database, we 'hash' it with bcrypt. Hashing turns 'mypassword123' into a long scrambled string like '\$2a\$10\$XJ8...'. Even if someone stole the database, they can't read your password.

■ Hosting: Vercel + Render

Vercel	Hosts the frontend (the React app). Every time you push to GitHub, Vercel automatically rebuilds and redeploys. Free forever for personal projects.
Render	Hosts the backend (the Node.js server). Also auto-deploys from GitHub. Free tier sleeps after 15 minutes of inactivity — first request after sleeping takes ~30 seconds to wake up. Perfectly fine for personal use.
GitHub	Stores your code. Acts as the bridge between your computer and both Vercel and Render. When you push code to GitHub, both platforms update automatically.

PART 3

How The Code Is Structured

Folder Structure

bond-app/ folder layout

bond-app/

■■■ backend/	← The server (Node.js)
■ ■■■ server.js	← Main file. Starts the server, defines all routes
■ ■■■ db.js	← Sets up the SQLite database and creates tables
■ ■■■ .env	← Secret keys (NOT uploaded to GitHub)
■ ■■■ .env.example	← Template showing what keys are needed
■ ■■■ package.json	← Lists all packages the backend needs
■ ■■■ routes/	
■ ■■■ auth.js	← (legacy) originally had login/register here
■ ■■■ memories.js	← Get, add, delete memories
■ ■■■ bucketlist.js	← Get, add, complete, delete bucket items
■ ■■■ ai.js	← Three AI endpoints using Google Gemini
■	
■■■ frontend/	← The website (React)
■■■ index.html	← The single HTML file React injects into
■■■ vite.config.js	← Vite settings (local proxy for /api)
■■■ package.json	← Lists all packages the frontend needs
■■■ src/	
■■■ main.jsx	← Entry point – mounts React onto index.html
■■■ App.jsx	← Router – decides which page to show
■■■ index.css	← All styling for the entire app
■■■ context/	
■ ■■■ AuthContext.jsx	← Global state: who is logged in?
■■■ components/	
■ ■■■ Sidebar.jsx	← Left navigation bar
■■■ pages/	
■■■ Landing.jsx	← Homepage (before login)
■■■ Auth.jsx	← Login & register form
■■■ AppShell.jsx	← Layout wrapper (sidebar + main area)
■■■ Dashboard.jsx	← Home screen after login
■■■ Memories.jsx	← Memories gallery
■■■ BucketList.jsx	← Bucket list
■■■ CheckIn.jsx	← Daily mood check-in
■■■ Support.jsx	← Support mode

How The Frontend Talks To The Backend

Every time you click something in the app that needs data — log in, add a memory, do a check-in — the frontend makes an HTTP request to the backend. This is called an API call. Here is what happens step

by step when you log in:

- **1.** You type your email and password and click 'Sign in'.
- **2.** The frontend runs: `fetch('/api/auth/login', { method: 'POST', body: { email, password } })`
- **3.** That request travels to the backend server on Render.
- **4.** `server.js` receives it at the route: `app.post('/api/auth/login', ...)`
- **5.** The backend checks the database: does this email exist? Does the password match?
- **6.** If yes, it creates a JWT token and sends back: `{ token, user }`
- **7.** The frontend saves the token in `localStorage` (like the browser's memory).
- **8.** Every future request sends that token in a header so the server knows who you are.

How The Database Works

The database has 5 tables. Think of each table as a sheet in a spreadsheet:

users	One row per person. Stores name, email, hashed password, invite code, avatar colour, friend_id (who they're bonded with), bond start date.
memories	One row per memory. Stores which bond it belongs to (bond_id), who created it, title, description, emoji, image URL, date.
bucket_list	One row per bucket item. Stores bond_id, title, category, whether it's completed, completion date.
checkins	One row per check-in. Stores bond_id, mood (1-5), the user's message, and the AI's response.
messages	Reserved for future chat feature. Currently not used in the UI.

How The AI Is Called

When you do a check-in, here is exactly what happens behind the scenes:

routes/ai.js — simplified check-in flow

1. Frontend sends: POST /api/ai/checkin { mood: 4, message: 'Good day!' }

2. Backend looks up the user and their friend from the database

3. Backend builds a prompt like:

'You are a friendship companion. User: Arjun. Friend: Priya.

Bonded 200 days. 10 memories. Mood today: Pretty good.

Message: Good day! Write a warm personal response...'

4. Backend sends that prompt to Google Gemini:

```
const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' })
```

```
const result = await model.generateContent(prompt)
```

5. Gemini replies with a warm paragraph of text

6. Backend saves the check-in to the database, returns response to frontend

7. Frontend displays the AI response on screen

PART 4

Every Error We Hit & How We Fixed It

This section explains every error that appeared during development. Understanding errors is one of the most valuable skills in programming — knowing *why* something broke is how you become a better developer.

Error 1 — Unexpected token 'T', 'The page c...' is not valid JSON

ERROR	<i>Unexpected token 'T', "The page c..." is not valid JSON</i>
CAUSE	The frontend was trying to call <code>/api/auth/register</code> . The server didn't recognise that route because of a broken routing trick in <code>server.js</code> — it was trying to manually rewrite the request URL using <code>req.url</code> . When Express couldn't find the route, it returned an HTML error page (which starts with 'The page cannot...'). The frontend then tried to parse that HTML as JSON and crashed.
FIX	Rewrote <code>server.js</code> to define all auth routes (register, login, me, connect) directly in <code>server.js</code> itself — no tricks, no URL rewriting. Also added a catch-all at the bottom that always returns JSON (never HTML) for unknown routes.

What 'JSON parse error' means in plain English

The frontend always expects the backend to reply with JSON — a format that looks like: `{ "token": "abc123", "user": { "name": "Arjun" } }`

When the backend returned an HTML page instead (starting with 'The page c...'), the frontend tried to read it as JSON and immediately crashed because 'T' (the first letter of 'The') is not how JSON starts. JSON always starts with `{` or `[`.

Error 2 — Node.js Version Incompatibility

ERROR *Installation error / app crashes on Node.js v22*

CAUSE The package better-sqlite3 is a 'native' package — it compiles C++ code during installation and must be rebuilt for each Node.js version. Node.js v22 had breaking changes that better-sqlite3 hadn't yet been updated to support, causing the build to fail.

FIX Downgraded to Node.js v20 (the LTS — Long Term Support — version). LTS versions are more stable because they are supported for years. Added a .nvmrc file containing '20' so the project always signals which version to use.

What LTS means

LTS stands for Long Term Support. Think of Node.js releases like phone models. v22 is the newest shiny phone with some bugs. v20 LTS is last year's model — fully stable, well tested, and supported for 3 more years. For real projects, always use the LTS version.

Error 3 — Failed to fetch (CORS error on Vercel)

ERROR *Access to fetch has been blocked by CORS policy*

CAUSE When running locally, Vite proxies /api requests to localhost:5000 — so the browser never sees a cross-origin request. On Vercel, that proxy doesn't exist. The frontend on vercel.app was calling the backend on onrender.com — two completely different domains. The browser's security rules (CORS) block cross-domain requests unless the server explicitly allows them.

FIX Two fixes together: (1) Changed AuthContext.jsx to use the VITE_API_URL environment variable (the real Render URL) instead of just '/api'. (2) Added the Render URL as VITE_API_URL in Vercel's environment variables settings.

What CORS is — explained simply

CORS stands for Cross-Origin Resource Sharing. 'Origin' means a domain+port combination. Your browser has a built-in security guard. When your frontend on vercel.app tries to fetch data from onrender.com, the guard stops it and says: 'Hey, are you allowed to talk to that other website?' The backend on Render must reply: 'Yes, I allow requests from vercel.app' by sending a special header called Access-Control-Allow-Origin. If the backend doesn't send that header, or sends the wrong value, the browser blocks the request before it even reaches your code — hence 'failed to fetch'.

Error 4 — CORS header has wrong Vercel URL

ERROR	<i>Access-Control-Allow-Origin header has value 'bond-app.vercel.app' not equal to supplied origin 'bond-6a57kykbx.vercel.app'</i>
CAUSE	Vercel gives every deployment TWO URLs: a permanent one (bond-app.vercel.app) and a preview one with a random code (bond-6a57kykbx-ibhagayarajis-projects.vercel.app). The FRONTEND_URL environment variable on Render only had the permanent URL. When the app loaded from the preview URL, the CORS check failed because the origin didn't match.
FIX	Updated server.js to allow any URL containing 'vercel.app' — not just one specific URL. This means all current and future Vercel deployment URLs for this project are allowed. Also kept localhost URLs allowed for local development.

The CORS fix in server.js

```
app.use(cors({
  origin: function (origin, callback) {
    if (!origin) return callback(null, true);          // Allow Postman/curl
    if (origin.startsWith('http://localhost'))        // Allow local dev
      return callback(null, true);
    if (origin.includes('vercel.app'))                // Allow ALL Vercel URLs
      return callback(null, true);
    callback(new Error('Not allowed by CORS'));      // Block everything else
  },
  credentials: true
}));
```

PART 5

Deployment — How It All Works Online

The Big Picture

When someone opens your app in their browser, here is the full journey of what happens:

- 1. User opens `bond-app.vercel.app` in their browser.
- 2. Vercel serves the React app (HTML + CSS + JavaScript files).
- 3. React loads in the browser. The user sees the landing page.
- 4. User clicks 'Sign in' and enters their details.
- 5. React sends a `fetch()` request to `https://bond-backend.onrender.com/api/auth/login`
- 6. Render wakes up (if sleeping) and processes the request in Node.js.
- 7. Node.js checks the SQLite database — is the email/password correct?
- 8. Render sends back a JSON response with the JWT token and user data.
- 9. React stores the token, updates the UI — user is now logged in.
- 10. User adds a memory — React sends another request to Render, Render saves to SQLite.
- 11. User does a check-in — Render calls Google Gemini API, gets response, returns it.

Environment Variables — What They Are

Environment variables are like secret settings that live outside your code. You never put them in code because code goes on GitHub (public). Instead you set them in the hosting platform's dashboard.

GOOGLE_API_KEY	Your Google Gemini API key. Kept secret — if someone else had this, they could use your free quota.
-----------------------	---

JWT_SECRET	A random string used to sign JWT tokens. If someone knew this, they could fake login tokens for any user.
-------------------	---

FRONTEND_URL	Set on Render. Tells the backend which frontend URL is allowed to make CORS requests. We broadened this to allow all <code>vercel.app</code> URLs.
---------------------	--

VITE_API_URL	Set on Vercel. Tells the frontend where the backend lives (e.g. <code>https://bond-backend.onrender.com/api</code>). <code>VITE_</code> prefix means Vite will make it available in the React code.
---------------------	--

Why GitHub Is In The Middle

GitHub is not just storage — it is the trigger for everything. When you run 'git push':

- GitHub receives your new code.
- Vercel detects the new commit and automatically rebuilds the frontend (takes ~1 min).
- Render detects the new commit and automatically rebuilds the backend (takes ~2 min).
- Both platforms update without you doing anything else.

This pattern is called CI/CD — Continuous Integration / Continuous Deployment. It's how every professional software team in the world ships code.

Render's Free Tier Sleep Behaviour

Why the app sometimes takes 30 seconds to respond

Render's free tier shuts down your server after 15 minutes of no traffic to save resources.

The next request 'wakes it up' — this takes about 30 seconds.

After it's awake, everything is fast again.

This is completely normal for free hosting. Paid plans (\$7/month on Render) keep it always awake.

For a personal friendship app, the free tier is perfectly fine.

PART 6

Questions Anyone Might Ask You

If you show this project to a teacher, interviewer, or fellow developer, here are the most common questions they'll ask — and exactly how to answer them.

Q1. What is the Bond app?

It's a full-stack web application for two close friends. It has shared memories, a bucket list, daily AI-powered check-ins, a support mode, and bond insights. Two friends connect using a private invite code and get a shared private space.

Q2. What tech stack did you use?

React with Vite for the frontend, Node.js with Express for the backend, SQLite for the database, Google Gemini 1.5 Flash for AI, and JWT for authentication. Hosted on Vercel (frontend) and Render (backend).

Q3. Why React and not plain HTML?

React makes it easy to build interactive UIs with reusable components. When the user adds a memory, React instantly updates the screen without refreshing the page. With plain HTML you'd have to reload the entire page every time something changed.

Q4. What is an API?

API stands for Application Programming Interface. It's how the frontend and backend talk to each other. The backend exposes URLs (called endpoints) like `/api/memories` or `/api/auth/login`. The frontend calls these URLs to get or save data.

Q5. How does authentication work?

When you log in, the server checks your password (hashed with `bcrypt`) against the database. If it matches, the server creates a JWT token — a signed encrypted string — and sends it back. The frontend stores this token and sends it with every request. The server verifies the token to know who you are.

Q6. Why SQLite instead of MongoDB or PostgreSQL?

SQLite requires zero setup — it's just a file (`bond.db`). For a personal app with two users, there's no need for a separate database server. MongoDB or PostgreSQL would be better choices if the app needed to scale to thousands of users.

Q7. What is CORS and why did you run into it?

CORS is a browser security rule that blocks requests between different domains. The frontend on `vercel.app` and the backend on `onrender.com` are different domains, so the browser blocked requests. We fixed it by telling the backend to explicitly allow requests from any `vercel.app` URL.

Q8. Why Google Gemini and not ChatGPT?

Google Gemini has a generous free tier — 15 requests per minute and 1 million tokens per day at no cost. OpenAI's ChatGPT API requires a paid account. For a personal project where keeping costs at zero is the goal, Gemini was the right choice.

Q9. How do you keep the API key secret?

The API key is stored as an environment variable on Render — never in the code. The `.gitignore` file ensures the `.env` file (which contains the key locally) is never uploaded to GitHub. Only the `.env.example` file (with placeholder values) is shared.

Q10. How does the invite/bonding system work?

When a user registers, the server generates a random 8-character invite code and stores it in the database. When another user enters that code, both users' records are updated with each other's ID as `friend_id`. A `bond_id` is computed as the smaller user ID + underscore + larger user ID (e.g. `'3_7'`) and used to link all shared data.

Q11. What happens when you push to GitHub?

Both Vercel and Render watch your GitHub repository. When you push new code, Vercel automatically rebuilds and redeploys the frontend, and Render rebuilds the backend. This is called CI/CD — Continuous Deployment.

Q12. Why does the app sometimes take 30 seconds to load?

Render's free tier puts the backend server to sleep after 15 minutes of inactivity. The first request wakes it up, which takes about 30 seconds. After that, it's fast. This is a known trade-off of free hosting.

Q13. Could you add more users / a group feature?

The current architecture only supports pairs of users. To support groups, you'd need to redesign the bonding system — instead of `friend_id` on each user, you'd create a separate 'bonds' or 'groups' table that multiple users can join.

Q14. How would you improve this app?

Several ways: add real image uploads using Cloudinary, add real-time messaging using Socket.io, add push notifications for check-in reminders, build a mobile version using React Native, add a mood graph over time using a charting library, or switch to PostgreSQL for more robust data storage.

Q15. What is a JWT token?

JSON Web Token. It is a string in three parts separated by dots: header.payload.signature. The payload contains your user ID. The signature is created using the JWT_SECRET key. Anyone can read the payload, but only the server can create a valid signature — so it cannot be faked.

Q16. What is bcrypt?

A library for hashing passwords. Hashing is a one-way process — you can turn 'mypassword' into a hash, but you cannot turn the hash back into 'mypassword'. When you log in, bcrypt hashes what you typed and compares it to the stored hash. This means even if the database were stolen, passwords would be safe.

Q17. Why is the frontend on Vercel and backend on Render?

Vercel is specialised for frontend/static site hosting — it has a global CDN, instant deployments, and is optimised for React. Render is designed for backend servers that need to run Node.js processes continuously. Using each platform for what it's best at gives the best performance for free.

Q18. What is the .env file?

A file that stores secret values locally (API keys, passwords, JWT secrets). It is listed in .gitignore so it is never uploaded to GitHub. The .env.example file shows which variables are needed without revealing the actual values.

Q19. What does npm install do?

npm (Node Package Manager) reads the package.json file and downloads all listed dependencies into a node_modules folder. It's like installing apps from an app store — you declare what you need, npm fetches it from the internet.

Q20. What is the difference between the frontend and backend?

The frontend is everything the user sees — the React code that runs in the browser. The backend is the server — the Node.js code that runs on Render, handles business logic, talks to the database, and calls external APIs like Gemini. They communicate with each other via HTTP requests (fetch calls).

Most people who want to build apps never start. You started, hit real errors, debugged them, and shipped a live product on the internet. That is the complete journey of a software developer.

26 files	written across frontend and backend
5 features	fully functional and AI-powered
4 real bugs	hit, debugged, and fixed
3 platforms	GitHub + Vercel + Render — all free
1 app	live on the internet, accessible to anyone

Made with ♥ for the friendships that matter most
